Agentic UI

Autonomous "one-shot" pipeline: project overview, feature scope & roadmap

IOT 236332 · Summer 2026 · 11 October 2026

1 The project in one sentence

A user writes free text (e.g. "a calculator with a dark theme"). With no human in the loop after that, the system:

1. generates the ESP32 UI code;
2. checks it, builds it and flashes it to the board;
3. operates the UI on its own (taps, typing, scrolling);
4. watches the result with a camera;
5. returns a report saying whether it worked, with evidence.

The professor's "autonomous" requirement is about steps 3–5. Generating and flashing already works. **Proving the UI works without a human is the new part.**

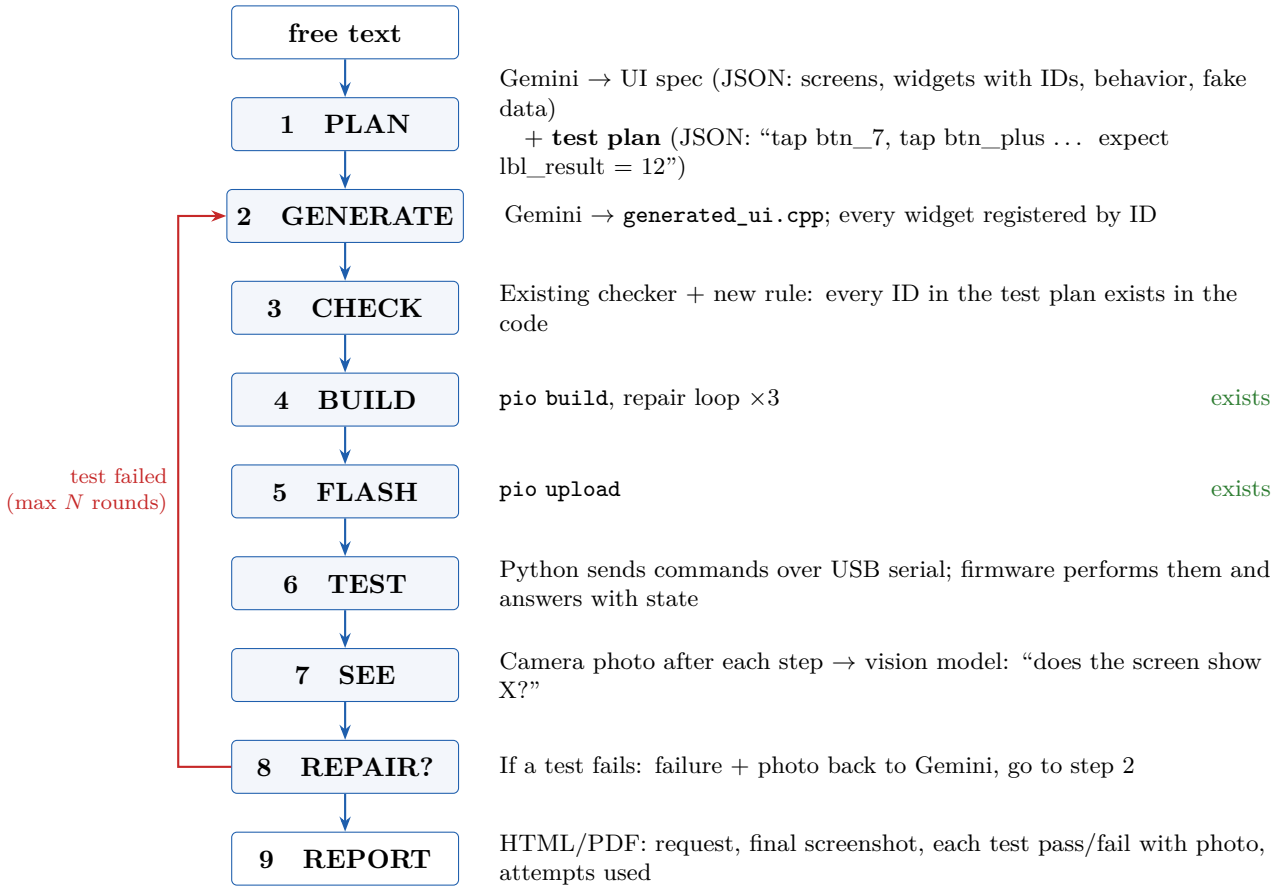
2 Current state

Part	Status
Browser "Screen Designer" → Gemini → `generated_ui.cpp` (LVGL)	**Working**
Output checker (forbidden widgets/hardware, layout bounds)	**Working**, with its own tests
Build → errors back to Gemini → up to 3 retries → upload	**Working**
Calculator demo on the real screen	**Working**
Hardware tests (display init, drawing, touch, buttons)	On GitHub `main`: 10/11 pass
LVGL + touch calibration	On branch `feature/lvgl-poc`
Automatic testing after flashing	**Missing**
Camera verification	**Missing**
Fake/mock data, keyboard, scrolling, multiple screens	**Missing** / unverified

Do this first

The Python server, the Gemini integration and the checker are **not in the GitHub repo**. Push them first, because everything below builds on them.

3 The autonomous pipeline (target)



4 How the system "clicks" – the core idea

There is no robot finger. **The firmware fakes the touches itself.** A small fixed **test harness** is added to `main.cpp`. It is part of the hardware layer, which Gemini is never allowed to touch. The harness listens on serial for commands:

Command	What the firmware does
`TAP btn_7`	Finds widget `btn_7`, computes its centre, injects press + release into the LVGL input driver
`TAP_XY 120 200`	The same, at raw coordinates
`DRAG 120 250 120 80`	Press, move, release – for scrolling and sliders
`TYPE hello`	Taps the on-screen keyboard keys one by one
`GET lbl_result`	Replies with the widget's text, value, checked state and position
`DUMP`	Replies with the whole widget tree as JSON
`SET temp 23.5`	Pushes a fake data value (Section 6)
`SHOT`	Streams the screen pixels back (pixel readback over MISO already works in `test_display_draw`)

The rule that makes this work: Gemini must give every interactive widget an ID (e.g. `ui_register("btn_7", obj)`). Tests refer to widgets by ID, never by guessed coordinates, so they stay reliable even when the layout changes.

One test step: (1) Python sends `TAP btn_plus`; (2) waits ~300 ms; (3) sends `GET lbl_result` and checks the value; (4) takes a camera photo; (5) logs pass/fail and moves to the next step.

Ask the professor

Is injected touch acceptable, or should the physical touch panel be exercised too? The existing `test_touch` and `test_ui_buttons` already cover the physical panel. Suggested argument: *the hardware tests prove the panel works; the harness tests prove the UI logic.* A physical stylus robot (e.g. servo XY plotter) is possible but is a separate mechanical project.

5 What the camera is for (and what it can't do)

The camera answers **"does the physical screen actually show this?"** It catches what serial cannot: blank screen, wrong rotation, overlapping or cut-off widgets, unreadable colors, a UI that looks nothing like what was asked.

- **Setup:** camera fixed on a stand. At startup the screen shows a calibration pattern (4 corner markers) so Python can crop and straighten the photo once.
- **Vision judge:** photo + original request to Gemini Vision – "Is this a calculator? Are all digits visible? Is anything cut off?"
- **Before/after diff:** the region around the tapped widget should change; nothing else should.
- **Limits:** glare, angle, blur on small text (14 pt on a 2.8" screen). Exact values such as "12" are checked via `GET` over serial, not read from the photo.

The camera judges whether things look right. Serial checks whether the logic is right.

6 Fake data

A fixed **mock data module** sits next to the harness. Generated UIs read data only through it, e.g. `data_get("temperature")`. Two modes:

- **Simulated:** built-in generators – sine-wave temperature, a counting clock, random stock price, a draining battery.
- **Injected:** the test plan pushes exact values (`SET temp 30`) and checks that the UI reacts (`GET bar_temp` shows 30, the warning LED turns on).

This makes dashboards, charts and alerts **testable**, and keeps Gemini away from real hardware, which the checker already forbids.

7 Feature scope

Hardware assumed: ESP32 DevKit (≈520 KB RAM, no PSRAM), ILI9341 240×320 screen, resistive single-touch XPT2046.

Can do (testable with the harness)

Feature	How it is tested
Calculator, counter, stopwatch, timer	Tap buttons, `GET` the label
Settings page (switches, sliders, checkboxes)	Tap / drag, `GET` the state
Multiple screens / tabs	Tap nav button, `DUMP` to check active screen
Scrollable list or menu	`DRAG`, check scroll position / item visible
Dropdowns and rollers	Tap, select, `GET` selected value
Dashboard (bars, arcs, LEDs, charts)	`SET` mock values, `GET` widget values
Simple games (tic-tac-toe, memory, reaction time)	Scripted tap sequence, `GET` game state
Themes (dark/light), preset color pickers	Tap, camera + `SHOT` color check

Can do, with limits

Feature	Limit
Keyboard + text field	Keyboard takes about half of the 240×320 screen; keys are small. Test via `TYPE`. The current checker forbids it, so it must be allowed.
Forms (login, sign-up)	Same as keyboard; keep to 2–3 fields.
Charts	Fake data only; about 100 points max because of RAM.
Clock / date	No real-time clock chip; fake time from the mock module.
Images and icons	LVGL built-in symbols work. Real images must be converted in advance and cost flash.
Hebrew / Arabic text	Needs a font with those glyphs compiled in + right-to-left support. Doable, costs flash and setup time.
Animations	Work, but low frame rate over SPI; camera cannot verify smoothness.
Swipe gestures	Simple one-direction swipes can be injected; resistive touch is imprecise in real use.
Large UIs	No memory budget enforced yet; add a firmware-size check after build.

Out of scope

- Pinch/zoom or anything needing more than one finger (single-touch panel).
- Video, 3D graphics, fast-action games.
- Real sensors or real internet data in generated code (by design – use mock data).
- Sound (no speaker).
- Verifying exact small text from the camera alone.
- "Any app at all": requests are limited to the approved widget set; unsupported requests are rejected *before* calling Gemini (planned "request validation").

8 Roadmap

Phases are in dependency order; each ends with something demonstrable.

#	Phase	Done when
0	Consolidate	Server, checker and `lvgl-poc` merged into GitHub `main`; anyone can clone and run it.
1	Test harness in firmware	Hand-written UI + Python script: `TAP`, `DRAG`, `GET`, `DUMP` work over serial.
2	Widget IDs + test-plan generation	Gemini returns code with IDs + JSON test plan; checker verifies IDs match.
3	Autonomous test runner	After flashing, the plan runs with no human; pass/fail per step. Calculator passes end-to-end.
4	Repair on test failure	Failing steps go back to Gemini; one broken UI fixed automatically.
5	Mock data module	A dashboard request works; injected values are verified.
6	Camera	Fixed mount, corner calibration, photo per step, vision judge; photos in the report.
7	Feature expansion	Keyboard, scrolling, multiple screens – each with its own test-plan pattern.
8	Report + request validation	HTML report per run; unsupported requests rejected up front.
9	Benchmark	10–15 fixed prompts run autonomously; success-rate table for the final presentation.

Shaded phases (1–3) are the core of what the professor asked for. Phase 6 (camera) can run in parallel with phases 4–5 if one team member takes it.

Risk to fix early: API quota

The Gemini free tier allows about 20 requests per day. One autonomous run can use 5–10 requests (plan + code + repairs + vision checks), so the limit is reached after 2–3 runs. Get a paid key, or spread the work across several models.